



SQL window functions

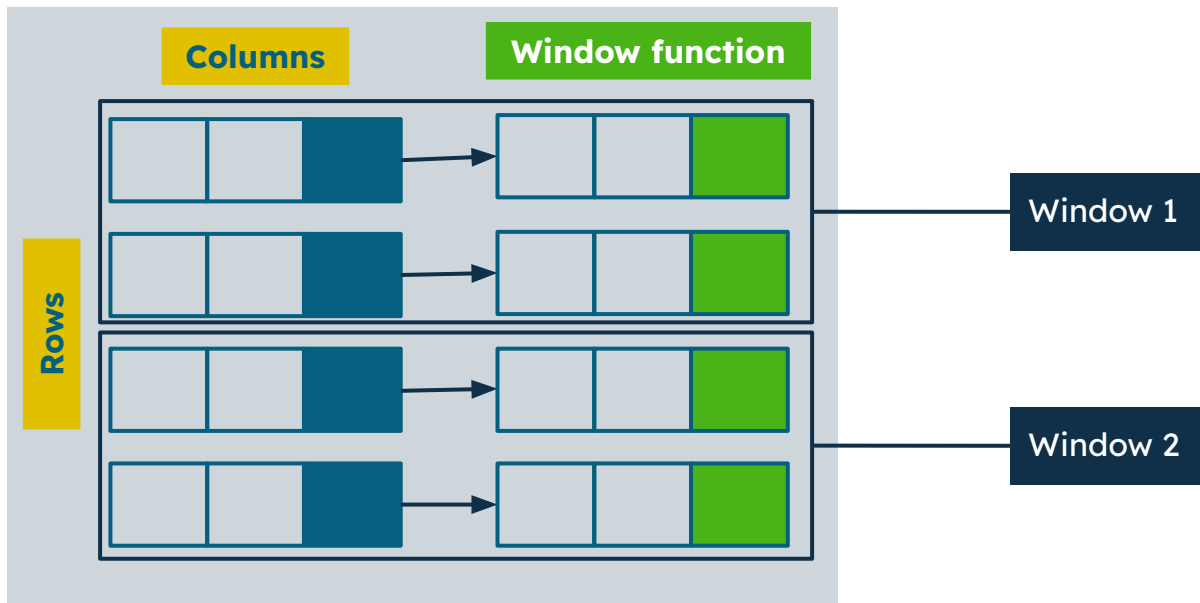
Window functions in SQL

Please do not copy without permission. © School of Entrepreneurship and Technology.



Window functions in SQL

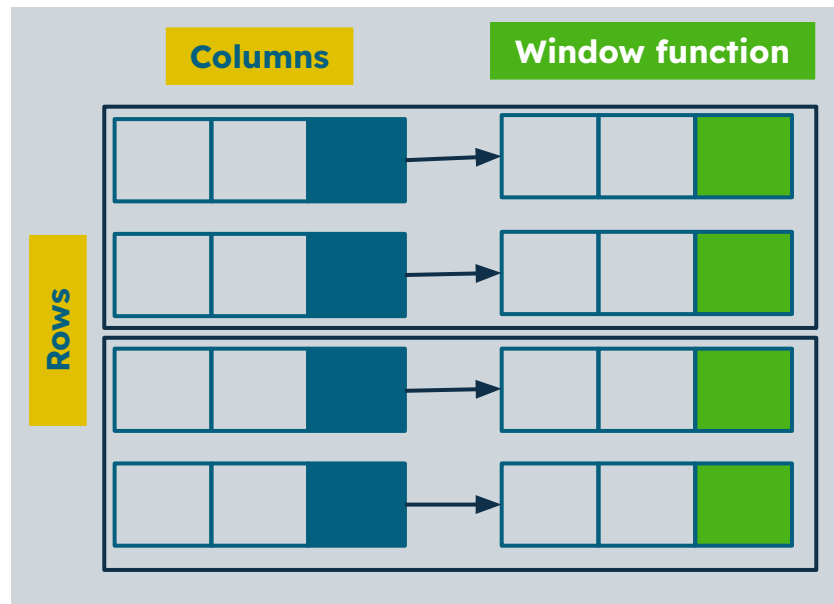
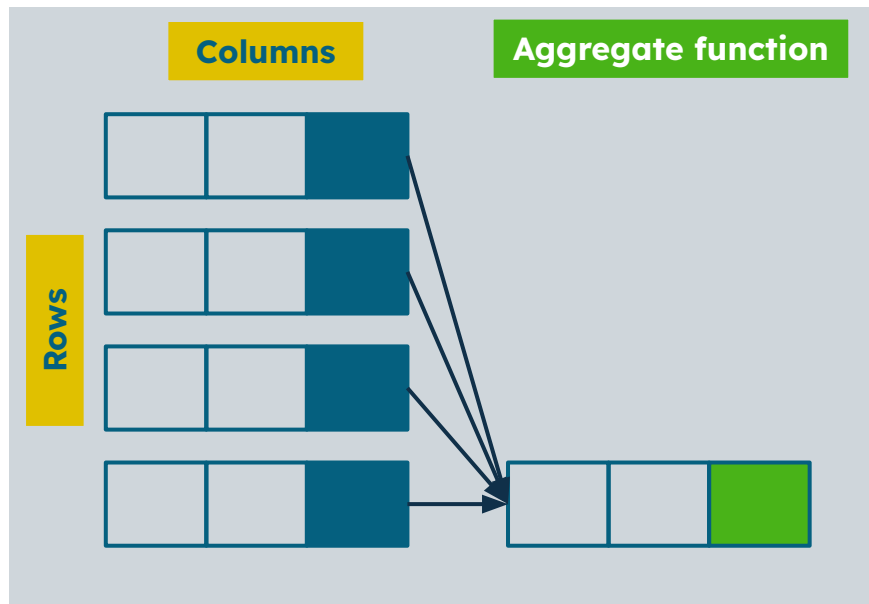
Window functions perform **ranking, aggregation, and value operations** on a **set of rows called a window**, and they return a **value for each row** in the set. A window can be made up of **1 or more rows**. A column can be partitioned into **1 or more windows**. If a column has one window, then all rows within that column are within the window.





Window functions in SQL

Window functions are contrary to aggregate functions, which only return a **single value from each column** they operate on. There is always a result on each row, as illustrated in blue below.





Importance of window functions in SQL

01. Efficiency

They allow **complex calculations and aggregations in a single query** rather than executing separate queries or using procedural logic, thus increasing performance.

02. Data Partitioning

They allow division of data into logical partitions based on specified criteria. This enables calculations and aggregations to be performed separately within each partition, providing a **more granular analysis** of your data.

03. Reporting and visualization

They allow you to easily calculate **running totals**, **moving averages**, and other analytical metrics that provide insights into your data.

04. Flexibility

They provide flexibility in defining the window or subset of rows over which the calculations are performed.

05. Code simplicity

They simplify the SQL code by eliminating the need for temporary tables or subqueries to achieve similar results. This leads to **cleaner and more readable code**, making it easier to maintain and understand.



Syntax of window functions

```
SELECT column1, column2, ..., window_function()  
OVER (PARTITION BY columnX ORDER BY columnY)  
FROM  
table_name;
```

This is the specific window function we want to apply. It can be any supported SQL function, suchs SUM(), COUNT(), AVG(), RANK(), etc.

This keyword indicates that you are using a window function.

This clause **specifies the window** and divides the rows into separate **partitions or groups** based on the values in column X. The window function will be **applied independently** within each partition.

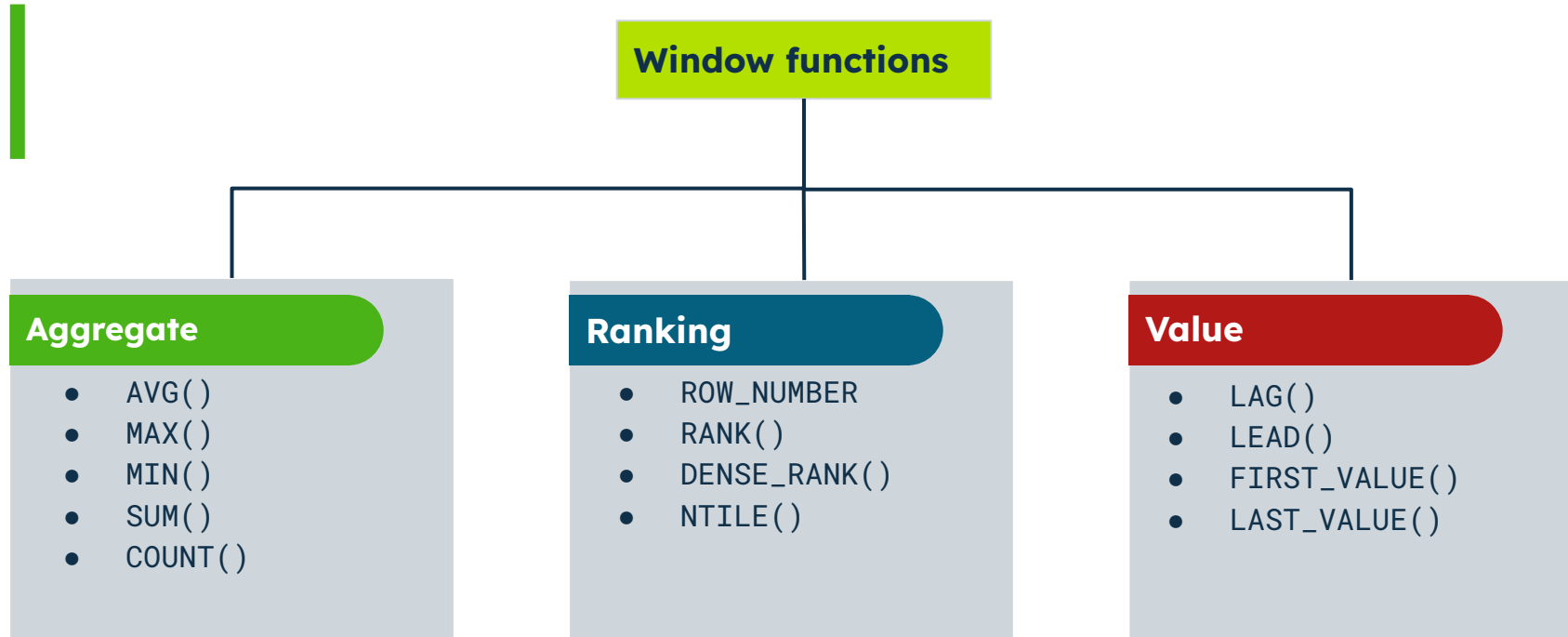
This clause determines the **order of rows within each partition**. It defines the logical order in which the window function operates.



The PARTITION BY and ORDER BY clauses can optionally be used together or separately in a query.



Types of window functions





Data overview (Employee table)

Start_date	Department	Province	First_name	Gender	Salary (Rand)
2015-06-01	Finance	Gauteng	Lily	Female	35760
2020-08-01	Marketing	Western_Cape	Gabriel	Male	30500
2022-03-01	Data_analytics	Free_State	Maryam	Female	46200
2022-07-15	Marketing	Gauteng	Sophia	Female	36900
2019-05-01	Data_analytics	Western_Cape	Alex	Male	36200
2012-01-01	Finance	Free_State	Martha	Female	48500
2014-05-01	Finance	Western_Cape	Joshua	Male	29500
2017-06-15	Data_analytics	Gauteng	Emily	Female	37800
2016-01-01	Marketing	Western_Cape	David	Male	31000

We will use the following table called “Employee” contains information about employees in a company.



Aggregate window functions

Aggregate window functions are a group of aggregate functions, such as `SUM()`, `COUNT()`, `AVG()`, `MAX()`, and `MIN()`, which are specifically applied to a certain window (set of rows).

Suppose we want to compare each employee's salary with the moving average salary of the corresponding department.

Query

```
SELECT Department,
       First_name,
       Salary,
       AVG(Salary) OVER (PARTITION BY Department
                        ORDER BY First_name) AS Average_salary
FROM Employee;
```

This is the aggregate window function that calculates the **moving average salary** of the "Salary" column within each partition defined by the "Department" column.

It then orders by 'First_name,' which is in ascending form.



Aggregate window functions (output)

Department	First_name	Salary	Average_salary
Data_analytics	Alex	36200	36200
Data_analytics	Emily	37800	37000
Data_analytics	Maryam	46200	40067
Finance	Joshua	29500	29500
Finance	Lily	35760	32630
Finance	Martha	48500	37920
Marketing	David	31000	31000
Marketing	Gabriel	30500	30750
Marketing	Sophia	36900	32800

01. Average salary between Alex and Emily

$$(36200 + 37800) / 2 = 37000$$

02. Average salary among Alex, Emily and Maryam

$$(36200 + 37800 + 46200) / 3 = 40067$$

03. The Average_salary column was ordered in ascending order by 'First_name' within the partition 'Data_analytics.'



Aggregate window functions

Suppose we want to compare each employee's salary with the average salary of the corresponding department.

Query

```
SELECT Department,  
       First_name,  
       Salary,  
       AVG(Salary) OVER (PARTITION BY Department)  
       AS Average_salary  
FROM  
       Employee;
```

The ORDER BY clause is excluded, which removes the moving average in this case. Meaning that the average will be calculated based on the whole set of rows within the partition.



Aggregate window functions (output)

Department	First_name	Salary	Average_salary
Data_analytics	Emily	01. 37800	40067
Data_analytics	Alex	36200	40067
Data_analytics	Maryam	46200	40067
Finance	Joshua	29500	37920
Finance	Lily	35760	37920
Finance	Martha	48500	37920
Marketing	David	31000	32800
Marketing	Gabriel	30500	32800
Marketing	Sophia	36900	32800

01.

We can now compare each salary against each department's average salaries.



Ranking window functions

Ranking window functions assign a rank or row number to each row within a specified window or subset of rows.

01. ROW_NUMBER()

02. RANK()

03. DENSE_RANK()

General ranking function syntax

```
SELECT Column_1, Column_2,  
       RANKING FUNCTION() OVER (PARTITION BY ColumnX s  
sdf    ORDER BY ColumnY)  
FROM  
      Table_name;
```



The ROW_NUMBER() function

A ROW_NUMBER() function assigns a **unique sequential number** to each row within a partition, regardless of the column values. No two rows can have the same row number within a division. Suppose we want to partition by Province and label each row within the partition by First_name.

Query

```
SELECT First_name, Province,  
       ROW_NUMBER() OVER (PARTITION BY Province ORDER BY First_name) AS  
       Rank_assignment  
FROM  
     Employee;
```

This is the ranking window function that assigns a rank to each row within a province partition and orders it according to the 'Province' column.



The ROW_NUMBER() function (output)

First_name	Province	Rank_assignment
Martha	Free_State	01. 1
Maryam	Free_State	2
Emily	Gauteng	1
Lily	Gauteng	2
Sophia	Gauteng	3
Alex	Western_Cape	02. 1
David	Western_Cape	2
Gabriel	Western_Cape	3
Joshua	Western_Cape	4

01.

Sequential row assignment in the **Free_State** province partition.

02.

Sequential row assignment in the **Western_Cape** province partition.



The RANK() function

The RANK() function assigns a rank to each row based on the order specified within the window. **Rows with the same values receive the same rank, and the next rank is skipped**, which is contrary to the ROW_NUMBER() function. Suppose we want to rank each row by Province.

Query

```
SELECT First_name, Province,  
       RANK() OVER (ORDER BY Province) AS Rank_assignemnt  
FROM  
     Employee;
```

This is the ranking window function that assigns each row within a province partition with a rank and orders it according to the 'Province' column.

We use the 'ORDER BY' clause only because it **already groups/partitions the provinces** hence, rows will have the same values, and the **same rank** will be assigned.



The RANK() function (output)

First_name	Province	Rank_assignment
Martha	Free_State	1
Maryam	Free_State	1
Emily	Gauteng	3
Lily	Gauteng	3
Sophia	Gauteng	3
Alex	Western_Cape	6
David	Western_Cape	6
Gabriel	Western_Cape	6
Joshua	Western_Cape	6

5 rows

01.

Rows with the same values (Gauteng) receive the same rank (rank 1)..

02.

Rank 2 is skipped because 'Maryam' fell under rank 1.

03.

Rank 4 and 5 are skipped because there are **5 rows** above, even though some rows have the same provinces



The DENSE_RANK() function

The DENSE_RANK() function operates similarly to the RANK() function, except it does not skip any ranks even if rows have the same values.

Query

```
SELECT First_name, Province,  
       DENSE_RANK() OVER (ORDER BY Province) AS Rank_assignment  
FROM  
     Employee;
```

This is the ranking window function that assigns each row within a province partition with a rank and orders it according to the 'Province' column.

We use the 'ORDER BY' clause only because it **already groups/partitions the provinces**; hence, rows will have the same values, and the **same rank** will be assigned.



The DENSE_RANK() function (output)

First_name	Province	Rank_assignment
Martha	Free_State	1
Maryam	Free_State	1
Emily	Gauteng	2
Lily	Gauteng	2
Sophia	Gauteng	2
Alex	Western_Cape	3
David	Western_Cape	3
Gabriel	Western_Cape	3
Joshua	Western_Cape	3

No ranking skipped. This is the most popular ranking function because it accounts for row values that are in the same ranking while keeping a sequential order.



Value window functions

Value window functions return a result for each row within the window and can access the values of other rows within the window and perform calculations based on those values.

01. LAG()

02. LEAD()

03. FIRST_VALUE()

04. LAST_VALUE()

General value function syntax

```
SELECT Column_1, Column_2,  
       RANKING FUNCTION() OVER (PARTITION BY ColumnX  
sdf   ORDER BY ColumnY)  
FROM  
      Table_name;
```



The LAG() function

The LAG() function allows access of a value within the column of interest from a previous row within the window frame. Suppose we want to retrieve the previous salaries according to the employee's date of hire.

Query

```
SELECT Start_date, First_name, Salary,  
       LAG(Salary) OVER (ORDER BY Start_date) AS Previous_salary  
FROM  
     Employee;
```

The LAG(Salary) function is applied to the salary column and ordered by Start_date. It returns the salary from the previous row as the column 'Previous_salary'.



The LAG() function (output)

Start_date	First_name	Salary	Previous_salary
2012-01-01	Martha	48500	NULL
2014-05-01	Joshua	29500	48500
2015-06-01	Lily	35760	29500
2016-01-01	David	31000	35760
2017-06-15	Emily	37800	31000
2019-05-01	Alex	36200	37800
2020-08-01	Gabriel	30500	36200
2022-03-01	Maryam	46200	30500
2021-07-15	Sophia	36900	46200

The value on the first row will always be NULL since its previous value does not exist

The 'Previous_salary' column shows the the **value in the previous row** from that particular row.



The LEAD() function

The LEAD() function allows access to the value of a column from a **following row** within the window frame. It is the counterpart of the LAG() function. Suppose we want to retrieve the next employee's name according to the employee's date of hire.

Query

```
SELECT Start_date, First_name,  
       LEAD(First_name) OVER (ORDER BY Start_date) AS Next_employee  
FROM Employee;
```

The LEAD(First_name) function is applied to the 'First_name' column and ordered by Start_date. It returns the 'First_name' from the previous row as the column 'Previous_employee'.



The LEAD() function (output)

Start_date	First_name	Next_employee
2012-01-01	Martha	Joshua
2014-05-01	Joshua	Lily
2015-06-01	Lily	David
2016-01-01	David	Emily
2017-06-15	Emily	Alex
2019-05-01	Alex	Gabriel
2020-08-01	Gabriel	Maryam
2022-03-01	Maryam	Sophia
2021-07-15	Sophia	NULL

The 'Previous_salary' column shows the the **value in the previous row** from that particular row.

The value on the first row will always be NULL since its previous value does not exist

In this case the **whole column is the window** because we **did not partition the rows** or use 'PARTITION BY' clause.



The FIRST_VALUE() function

The FIRST_VALUE() function allows the retrieval of the value of a column from the **first row** within a window frame. Suppose we want to retrieve the first employee the company hired.

Query

```
SELECT Start_date, First_name,  
       FIRST_VALUE(First_name) OVER (ORDER BY Start_date) AS First_employee  
FROM  
     Employee;
```

The FIRST_VALUE(First_name) function is applied to the 'First_name' column and ordered by Start_date. It returns the 'First_name' from the first row as the column 'First_employee'.



The FIRST_VALUE() function (output)

Start_date	First_name	Next_employee
2012-01-01	Martha	Martha
2014-05-01	Joshua	Martha
2015-06-01	Lily	Martha
2016-01-01	David	Martha
2017-06-15	Emily	Martha
2019-05-01	Alex	Martha
2020-08-01	Gabriel	Martha
2022-03-01	Maryam	Martha
2021-07-15	Sophia	Martha

Only the first value or the first hired employee 'Martha' will be the output.

In this case the **whole column is the window** because we **did not partition the rows** or use the 'PARTITION BY' clause.



The LAST_VALUE() function

The LAST_VALUE() function allows the retrieval of the value of a column from the **last row** within a window frame. Suppose we want to retrieve the last employee the company hired based on their respective departments.

Query

```
SELECT Start_date, First_name,  
       LAST_VALUE(First_name) OVER (PARTITION BY Department) AS Last_employee  
FROM  
     Employee;
```

The FIRST_VALUE() function is applied to the 'First_name' column, partitioned by the department and ordered by Start_date. It returns the 'First_name' from the last row within a window.



The LAST_VALUE() function (output)

Start_date	Department	First_name	Next_employee
2017-06-15	Data_analytics	Emily	Maryam
2019-05-01	Data_analytics	Alex	Maryam
2022-03-01	Data_analytics	Maryam	Maryam
2012-01-01	Finance	Martha	Lily
2014-05-01	Finance	Joshua	Lily
2015-06-01	Finance	Lily	Lily
2016-01-01	Marketing	David	Sophia
2020-08-01	Marketing	Gabriel	Sophia
2022-07-15	Marketing	Sophia	Sophia

Within the 'Data Analytics' department, **Maryam** was the last employee to be hired on the date '2022-03-01.'

Within the 'Marketing' department, **Sophia** was the last employee to be hired on the date '2022-07-15.'